

PROPOSTA DE IMPLEMENTAÇÃO

# App Estágio CEFET-MG

**Domínio → Testes (TDD) → Armazenamento Permanente**

Como a IA vai construir e testar cada camada — Value Objects, Entities, Use Cases, Context API e expo-secure-store — antes de tocar em banco de dados.

*DDD • Clean Architecture • TDD • Jest + jest-expo + RNTL*

# O que vamos ver

---

01 Por que domínio primeiro

03 Entities & Aggregates

05 TDD: red → green → refactor

07 Mocks, fakes e stubs

09 expo-secure-store

11 Ordem de construção (checklist)

02 Value Objects

04 Use Cases (aplicação)

06 Jest + jest-expo + RNTL

08 Autenticação + Context API

10 Pirâmide de testes do projeto

# Domínio primeiro. Sempre.

*Nunca começar pelo schema do banco ou pela tela.*



- Sem import de expo-sqlite, expo-camera, expo-location, expo-print ou @supabase/supabase-js dentro de domain/ ou application/.
- Regra de dependência: camada externa depende da interna, nunca o contrário.
- Se um arquivo em domain/ ou application/ importar SDK direto → violação: mover a chamada para o adapter e expor só a interface.

# Value Objects

*Objeto imutável, sem identidade própria, validado na criação.*

- Criterio — nome + nota 0-10, valida range e calcula faixa MB/B/R/F
- Coordenada — lat/lng + timestamp
- Assinatura — base64 + timestamp
- StatusSincronizacao — pending | synced | error
- StatusPeriodo — enum de negócio do ciclo de vida
- CargaHoraria — total/período/mínima; valida cumprida  $\geq$  mínima

```
// domain/value-objects/Criterio.ts
export class Criterio {
  private constructor(
    readonly nome: string,
    readonly nota: number,
  ) {}

  static criar(nome: string, nota: number) {
    if (nota < 0 || nota > 10) {
      throw new Error('Nota fora de 0-10');
    }
    return new Criterio(nome, nota);
  }

  get faixa(): 'MB'|'B'|'R'|'F' {
    if (this.nota >= 9) return 'MB';
    if (this.nota >= 7) return 'B';
    if (this.nota >= 5) return 'R';
    return 'F';
  }
}
```

# Testando um Value Object primeiro

1. RED

2. GREEN

3. REFACTOR

```
// domain/value-objects/__tests__/Critério.spec.ts
describe('Critério', () => {
  it('rejeita nota fora de 0-10', () => {
    expect(() => Critério.criar('Pontualidade', 11))
      .toThrow('Nota fora de 0-10');
  });

  it('classifica nota 9.5 como faixa MB', () => {
    const c = Critério.criar('Pontualidade', 9.5);
    expect(c.faixa).toBe('MB');
  });
});
```

```
// RED: teste escrito antes da classe existir -> falha
// GREEN: implementa Critério.criar() só o suficiente para passar
// REFACTOR: extrai tabela de faixas, mantém teste verde
```

# Entities, Aggregate & Aggregate Root

*Entidade tem identidade e ciclo de vida. Aggregate Root protege as invariantes de tudo que está dentro dele.*

**PeriodoAvaliacao**  
**(Aggregate Root)**

*só acessadas via periodo.registrarAvaliacaoSupervisor(...)*

AtividadesDes  
envolvidas

AvaliacaoSuper  
visor

AutoAvaliacao

AssinaturaEsta  
giario

AssinaturaOrie  
ntador

CapturaOcr

- **Estagio**

- raiz própria; PeriodoAvaliacao a referencia via estagiold — muda raramente

- **TokenSupervisor**

- raiz própria; segurança/auditoria, ciclo de vida independente

- **Invariante protegida no agregado**

- não aceita 2ª avaliação se já Aprovado
- podeGerarPdf() só true com as 3 assinaturas

# Entidade + teste de invariante

```
// domain/entities/PeriodoAvaliacao.ts
export class PeriodoAvaliacao {
  private status: StatusPeriodo;
  private avaliacaoSupervisor?:
    AvaliacaoSupervisor;

  registrarAvaliacaoSupervisor(
    av: AvaliacaoSupervisor
  ) {
    if (this.status === 'Aprovado') {
      throw new Error(
        'Período já aprovado'
      );
    }
    this.avaliacaoSupervisor = av;
  }

  podeGerarPdf(): boolean {
    return this.assinaturas
      .every(a => a.status === 'syncd');
  }
}
```

```
// domain/entities/__tests__/PeriodoAvaliacao.spec.ts
it('bloqueia 2a avaliacao se ja Aprovado', () => {
  const periodo = criarPeriodoAprovado();

  expect(() =>
    periodo.registrarAvaliacaoSupervisor(nova)
  ).toThrow('Período já aprovado');
});

it('podeGerarPdf so com 3 assinaturas synced', () => {
  const periodo = criarPeriodoComAssinaturas({
    estagiario: 'syncd',
    orientador: 'syncd',
    supervisor: 'pending',
  });

  expect(periodo.podeGerarPdf()).toBe(false);
});

// sem mock: teste 100% em memória, 100% puro
```

# Use Cases orquestram o domínio

*Use case não contém regra de negócio — chama entidades/VOs e fala com interfaces (repository/gateway).*

- RegistrarAtividadesUseCase
- AvaliarDesempenhoUseCase
- RealizarAutoAvaliacaoUseCase
- AssinarRelatorioUseCase
- AprovarRelatorioUseCase /  
DevolverRelatorioUseCase
- GerarPdfUseCase
- SincronizarFilaUseCase
- AutenticarUsuarioUseCase /  
AcessarViaTokenUseCase

```
// application/use-cases/RegistrarAtividadesUseCase.ts
export class RegistrarAtividadesUseCase {
  constructor(
    private repo: PeriodoAvaliacaoRepository,
    private camera: CameraGateway,
    private location: LocationGateway,
  ) {}

  async executar(input: Input) {
    const periodo = await this.repo.buscar(
      input.periodoId
    );
    periodo.registrarAtividades(input.dados);
    await this.repo.salvar(periodo);
  }
}

// repo/camera/location = INTERFACES do domínio
// implementação real fica no adapter
```

# Use case + fakes in-memory

```
// application/use-cases/__tests__/RegistrarAtividadesUseCase.spec.ts
class PeriodoAvaliacaoRepositoryFake implements PeriodoAvaliacaoRepository {
  private db = new Map<string, PeriodoAvaliacao>();
  async buscar(id: string) { return this.db.get(id)!; }
  async salvar(p: PeriodoAvaliacao) { this.db.set(p.id, p); }
}
```

```
it('grava local e enfileira quando sem rede', async () => {
  const repo = new PeriodoAvaliacaoRepositoryFake();
  const camera = new CameraGatewayFake();
  const location = new LocationGatewayFake({ permissaoNegada: false });
  const useCase = new RegistrarAtividadesUseCase(repo, camera, location);
```

```
  await useCase.executar(inputValido);
```

```
  expect(await repo.buscar(input.periodoId)).toBeDefined();
});
```

```
// cobre: caminho feliz, permissão de localização negada,
//          câmera sem localização (bloqueia anexo - RF06), sem rede
```

# Pirâmide de testes do projeto



*Câmera e geolocalização NUNCA são exercitadas de verdade em teste automatizado.*

# Jest + jest-expo + React Native Testing Library

- **jest-expo**

- preset oficial: configura transform, mocks de módulos nativos Expo

- **@testing-library/react-native (RNTL)**

- renderiza telas de verdade, consulta por texto/role, dispara eventos (fireEvent/userEvent)

- sem testar detalhe de implementação — testa o que o usuário vê/faz

- **Cobertura**

- meta: projeto totalmente testado — cada RF tem teste de aceitação

- matriz RF → Caso de Uso → Teste no checklist de rastreabilidade

```
// adapters/screens/__tests__/AssinaturaScreen.spec.tsx
import { render, fireEvent } from
  '@testing-library/react-native';

it('desabilita botao ate assinar', () => {
  const useCaseFake = new AssinarRelatorioUseCaseFake();

  const { getByText, getByTestId } = render(
    <AssinaturaScreen useCase={useCaseFake} />
  );

  expect(getByText('Confirmar')).toBeDisabled();

  fireEvent(getByTestId('canvas'), 'onOK', 'base64...');

  expect(getByText('Confirmar')).not.toBeDisabled();
});
```

# Mock, Fake e Stub — quando usar cada um

## FAKE

Implementação real e simplificada, em memória.

*Exemplo:*

Repository fake (Map em vez de SQLite) nos testes de use case.

## MOCK

Substitui um módulo/SDK externo e verifica chamada.

*Exemplo:*

```
jest.mock('expo-location', ...) /  
jest.mock('expo-camera', ...)
```

## STUB

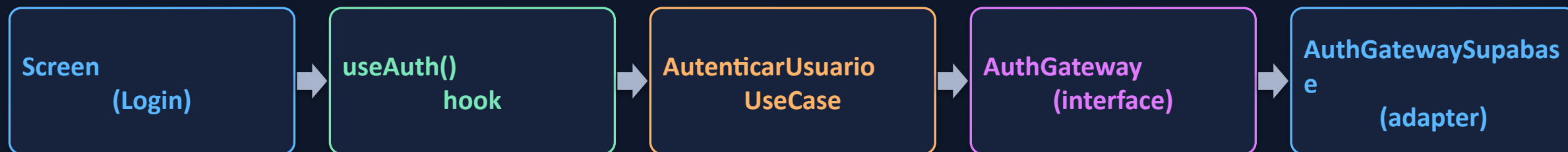
Retorna resposta fixa, sem lógica, sem verificação.

*Exemplo:*

AuthGateway stub devolvendo sessão válida para testar tela.

*TokenSupervisorRepositorySupabase é testado contra cenário de token expirado/revogado (mock de rede).*

# Como o login é implementado e testado



*Domínio nunca fala com @supabase/supabase-js diretamente — sempre pela interface AuthGateway.*

- 3 perfis com conta completa: Aluno, Professor/Orientador, Coordenação
- Supervisor NÃO tem conta Auth — acessa via TokenSupervisor (token de acesso, expiração semestral, revogável)
- Testado: AutenticarUsuarioUseCase com AuthGateway fake (credenciais válidas/inválidas por perfil); AcessarViaTokenUseCase (token válido/expirado/revogado)

# AuthContext: sessão dispon

- **Por que Context API**

- evita prop drilling do usuário logado por todas as telas

- useAuth() é hook fino de adapters/ — adapta o use case ao ciclo de vida do componente, não substitui o use case

- **sessionStorage aqui = wrapper sobre expo-secure-store**

- Context não sabe que é SecureStore — só chama a interface

- **Como é testado**

- RNTL:  
render(<AuthProvider><Tela/></AuthProvider>) com useCase fake

- confirma: sessão restaurada no boot, logout limpa contexto

```
// adapters/auth/AuthContext.tsx
const AuthContext = createContext<AuthContextValue | undefined>(undefined);

export function AuthProvider({ children }) {
  const [session, setSession] =
    useState<Sessao | null>(null);

  useEffect(() => {
    sessionStorage.carregar()
      .then(setSession);
  }, []);

  const login = async (cred) => {
    const s = await autenticarUseCase
      .executar(cred);
    await sessionStorage.salvar(s);
    setSession(s);
  };

  return (
    <AuthContext.Provider
      value={{ session, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
}
```

## ARMAZENAMENTO SEGURO DE SESSÃO

# expo-secure-store

### *Sessão de Aluno/Professor/Coordenação — nunca As*

- Criptografado no dispositivo (Keychain iOS / Keystore Android)
- Fica em adapters/ — implementa interface SessionStorage do domínio/aplicação
- Última camada antes de tocar armazenamento permanente real (SQLite/Supabase)

```
// __tests__/SessionStorageSecureStore.spec.ts
jest.mock('expo-secure-store');
it('salva e recupera sessao', async () => {
  await storage.salvar(sessaoValida);
  expect(await storage.carregar())
    .toEqual(sessaoValida);
});
```

```
// adapters/auth/SessionStorageSecureStore.ts
import * as SecureStore from 'expo-secure-store';

export class SessionStorageSecureStore
  implements SessionStorage {

  async salvar(sessao: Sessao) {
    await SecureStore.setItemAsync(
      'session', JSON.stringify(sessao)
    );
  }

  async carregar(): Promise<Sessao|null> {
    const raw = await SecureStore
      .getItemAsync('session');
    return raw ? JSON.parse(raw) : null;
  }

  async limpar() {
    await SecureStore.deleteItemAsync(
      'session'
    );
  }
}
```

# Tudo testado com IA antes do armazenamento permanente

---

|   |                                              |                                                                     |
|---|----------------------------------------------|---------------------------------------------------------------------|
| 1 | Value Objects                                | Criterio, Coordenada, Assinatura, CargaHoraria — testes puros       |
| 2 | Entities + Aggregates                        | PeriodoAvaliacao, Estagio, TokenSupervisor — invariantes testadas   |
| 3 | Domain Services                              | RegraGeracaoPdfService, RegraDevolucaoService, SincronizacaoService |
| 4 | Repository / Gateway (interfaces)            | contratos no domínio, sem implementação ainda                       |
| 5 | Use Cases                                    | orquestram domínio com fakes in-memory (red-green-refactor)         |
| 6 | Context API + hooks (useAuth, useAtividades) | liga use case ao ciclo de vida do componente                        |
| 7 | Telas (RNTL)                                 | AssinaturaScreen, AtividadesFormScreen — use case fake injetado     |
| 8 | expo-secure-store                            | sessão local segura — mockado em teste                              |

# Projeto totalmente testado

Cada RF tem teste de aceitação. Cada camada é validada com IA — de forma isolada e incremental — antes de qualquer dado tocar armazenamento permanente.

Domínio → Use Cases → Context API/expo-secure-store → SQLite/Supabase